

C-Agent 补充材料

CS2313 操作系统课程设计

Hengtao Wu 524031910038

June 2026

1 说明

主报告受五页篇幅限制，只能保留核心架构、关键设计和主要结论。本补充材料用于展示 handout 之外的工程化实现、运行截图和设计细节。它不是主报告的替代，而是对额外工作量和系统完整性的补充证据。

2 扩展功能总览

功能	补充说明
会话持久化	append-only log、checkpoint、恢复界面、重命名/删除命令
项目记忆	六类 typed memory files, remember/recall/update/delete 工具, 手动整理流程
Prompt 构造	基础 system prompt、选择性 memory 注入、skill manifest、上下文恢复提示
Skill 系统	Progressive disclosure: 启动时只注入 manifest, 完整 SKILL.md 按需加载
SubAgent	后台子智能体、独立 context、spawn/status/wait 工具
联网工具	web_fetch、web_search、private network 阻断、RSS 解析
Hooks	HOOK_BEFORE_TOOL/HOOK_AFTER_TOOL, memory observation hook
终端 UI	Spinner、工具状态、同步输出、MD4C Markdown 渲染

3 磁盘状态组织

C-Agent 的一个重要工程选择是：运行状态没有被粗暴地塞进一个不透明的大文件里。会话历史、被卸载的工具输出、长期记忆和技能包各自拥有独立目录结构和恢复语义。这样既便于人工检查，也让每个子系统有清晰的所有权边界。

Listing 1: .agent/ 下的项目级状态组织

```
.agent/  
  sessions/  
    <session_id>/  
      meta.json           # name, model, timestamps, message count  
      checkpoint.json     # full message snapshot, written atomically  
      log.jsonl           # append-only messages after latest checkpoint  
  
  offload/  
    <session_id>/  
      MANIFEST.md         # index of offloaded artifacts  
      0.txt  
      1.txt               # full tool outputs recoverable by read_file  
  
  memory/  
    MEMORY.md             # human-readable index  
    preference.md         # injected into future system prompts  
    fact.md               # injected into future system prompts  
    architecture.md       # queried on demand  
    workflow.md  
    pattern.md  
    bug.md  
  
  skills/  
    <skill-name>/
```

```
SKILL.md          # loaded on demand by load_skill
scripts/         # optional helper files
references/
assets/
```

目录	设计理由
sessions/	把可持久化的对话历史与临时运行状态分离。checkpoint+log 结构让系统具备崩溃恢复能力，同时避免每追加一条消息都重写整个会话。
offload/	类似上下文的“page-out”区域。大工具输出从 prompt 中移出，但仍可通过 MANIFEST.md 和 read_file 无损恢复。按 session 隔离可以避免不同对话的 artifact 混在一起。
memory/	用 typed Markdown 文件存储长期知识，而不是一个无结构 blob。这支持选择性 prompt 注入：默认只注入 preference 和 fact，其他 architecture/workflow/bug 信息通过 recall 按需查询。
skills/	把任务专用指令移出基础 prompt。启动时只暴露紧凑的 skill manifest，完整 SKILL.md 只有在模型判断相关时才加载。

这个文件组织是刻意设计成可检查的：用户可以直接打开目录，理解 agent 记住了什么，恢复某个 offload 输出，或者删除旧 session，而不需要数据库或专门 viewer。

4 终端 UI 与 Markdown 渲染

handout 只要求 live TUI、spinner 和工具状态。C-Agent 在此基础上额外集成了 MD4C，在终端中渲染最终回复的 Markdown。标题、代码块、列表、表格以及中英文混排都会更接近成熟 CLI 工具的输出体验。

```
>_zsh  c-agent  main ↑44 ↓1 ?3 ~1  1m 8s 699ms
>> ./start.sh
| ... (397 more lines)
以下是 C Agent 的 README 总结 📌

🧠 C Agent - 纯 C 实现的终端编程智能体

| 上海交通大学 CS2313 操作系统课程设计项目

🌟 核心亮点
```

特性	说明
纯 C11 实现	零 Python/JS 依赖，手搓 malloc/free + pthread
ReAct 推理循环	思考 → 行动 → 观察，多轮调用工具直到任务完成
16 个内置工具	bash、文件读写编辑、记忆管理、子智能体、联网搜索等
并发执行	只读工具用 pthread 自动并行调度
上下文管理	大输出自动卸载到磁盘 + LLM 摘要压缩历史
会话持久化	append-only 日志 + checkpoint，崩溃可恢复
项目记忆系统	6 类 typed files (fact/preference/architecture 等)，跨会话持久化
沙箱安全	路径限制 + 危险命令过滤
技能系统	按需加载 SKILL.md，避免 system prompt 过长
终端 UI	实时 spinner + Markdown 渲染

Figure 1: 终端中的 Markdown 渲染效果

5 会话持久化展示

每个 session 都存储在 `.agent/sessions/<id>/` 下,包含 `meta.json`、`checkpoint.json` 和 `log.jsonl`。REPL 还提供 session browser 和斜杠命令,用于新建、恢复、重命名和删除会话。

```
/session
Sessions 1-25/25  ↑ move  Enter restore  q/ESC cancel
* 你好呀 请你读一下readme 然后总结一下 用漂亮的md输出的
你会算?
记住
不要 我要你编一句话 里面开头正好是这个顺序
你上网找一下Hengtaoku的Github
有道理 你想成长为能操控GUI的或者CLI从而操控我的电脑的agent吗
why
不是格子图案 是鸽子 我打错了
你新增一个HEARTBEAT.md吧 每天晚上到十点钟提醒 但是我不知道怎么触发的
但是会429
这你还查什么 哈哈哈哈哈 你不笨啦就知道嘛
第三个点哦!
你是什么模型
我做好啦! 你可以diff看看的
我在测试你的表格md渲染的功能 你输出几个给我看看的
你好
给我输出大量的md格式
懒蛋蛋 输出markdown表格给我看看
你输出一个反引号和数字给我看看
没看到重新说一下
没看明白!
你好呀 教我markdown怎么写的
给我下单
</rss>
好呀
* current session
```

Figure 2: 交互式 session 恢复菜单

```
sessions
  > 20260531_112514
  > 20260531_124610
  (1) checkpoint.json
  (1) log.jsonl
  (1) meta.json
  > 20260531_124918
  > 20260531_130549
  > 20260531_133250
  > 20260531_133642
  > 20260531_133830
  > 20260531_134555
  > 20260531_134559
```

Figure 3: 包含 `meta.json`、`checkpoint.json` 和 `log.jsonl` 的 session 目录

6 Memory 与 Skill workflow

Memory 系统把长期项目知识拆成六类 typed files: `pattern`、`preference`、`architecture`、`bug`、`workflow` 和 `fact`。Agent 可以调用 `remember` 写入记忆,用 `recall` 查询,并通过 `memory_delete/memory_update` 维护条目。`/memory` 命令还会运行一个受限 `consolidator` prompt, 把工具观察记录提炼成最多五条高价值 memory。

```
C-AGENT
  > .agent
    > memory
      > architecture.md
      > bug.md
      > fact.md
      > MEMORY.md
      > pattern.md
      > preference.md
      > workflow.md
    > offload
    > sessions
    > skills
    > .codegraph
    > agent
    > build
```

```
.agent > memory > MEMORY.md > abc # Project Memory
1  # Project Memory
2
3  - [Patterns](pattern.md) - Coding patterns and conventions
4  - [Preferences](preference.md) - User preferences and interaction style
5  - [Architecture](architecture.md) - Project architecture and module responsibilities
6  - [Bugs](bug.md) - Known bugs, risks, and fixes
7  - [Workflows](workflow.md) - Build, test, run, and operational workflows
8  - [Facts](fact.md) - General project facts
9
```

Figure 4: typed project memory files

Skill 系统使用 progressive disclosure。启动时 Agent 只看到 skill 名称和一句话描述；当某个 skill 相关时，再调用 `load_skill` 读取完整 `SKILL.md`。这样既保持默认 prompt 紧凑，又允许任务专用行为按需展开。（这里展示的是我写的 demo-skill，里面就是指导 agent 展示 demo 的一些提示词）

```
zsh c-agent main +44 i1 ?3 ~1 0ms base 3.13.13 46% 12,10:54
>> ./start.sh
C Agent - /Users/pluto/CODE/course/c-agent
Model: deepseek-chat
Type 'exit' to quit.
> 请使用 c-agent-demo 技能。我需要附录截图：依次展示 Markdown 渲染、会话存储（session storage）、内存文件、卸载清单（offload manifest）、提示词构建以及运行时安全性。
✓ load_skill {"name":"c-agent-demo"} (0.0s)
-----
name: c-agent-demo
description: Use when presenting or demonstrating this C-Agent project, especially for course defense demos, feature walkthroughs, ...
allowed-tools: read_file, bash, recall, current_time
-----
# C-Agent Demo Skill
... (80 more lines)
✓ bash {"command":"ls -la .agent/sessions/ | head -20"} (0.0s)
total 0
drwxr-xr-x@ 36 pluto  staff  1152  6月 12 10:55 .
drwxr-xr-x@  6 pluto  staff   192  5月 31 12:54 ..
drwxr-xr-x@  5 pluto  staff   160  5月 31 11:33 20260531_112514
```

Figure 5: skill manifest 与按需加载

7 SubAgent 委托执行

`subagent_spawn`、`subagent_status` 和 `subagent_wait` 让主 Agent 可以把独立子任务委托给后台 child agent。每个 child agent 拥有自己的 context window，并以 silent 模式运行，因此中间探索过程不会污染父 Agent 的上下文。父 Agent 最终只接收结构化结果。

```
zsh c-agent main +44 i1 ?3 ~1 3m 12s 424ms base 3.13.13 45% 12,10:58
>> ./start.sh
C Agent - /Users/pluto/CODE/course/c-agent
Model: deepseek-chat
Type 'exit' to quit.
> 你去用两个subagent分别 读一下readme和context 然后汇总报告给我
. subagent_spawn {"task":"读取项目根目录下的 README.md 文件，把完整内容返回给我。"}
[subagent_2] starting: 读取项目根目录下的 context.md 文件（如果有的话），以及 .agent/ 目录下的 context.md 或类似的上下文文件。说明一下找到了哪些文件，并返回它们的内容。如果都没找到，也请说明。
✓ subagent_spawn {"task":"读取项目根目录下的 README.md 文件，把完整内容返回给我。"} (0.0s)
| subagent_1 started: 读取项目根目录下的 README.md 文件，把完整内容返回给我。
✓ subagent_spawn {"task":"读取项目根目录下的 context.md 文件（如果有的话），以及 .agent/ 目录下的 c..."} (0.0s)
| subagent_2 started: 读取项目根目录下的 context.md 文件（如果有的话），以及 .agent/ 目录下的 context.md 或...
x subagent_wait {"id":1} (0.0s)
| cannot open 'context.md': No such file or directory
x subagent_wait {"id":2} (0.0s)
| cannot open '.agent/context.md': No such file or directory
[subagent_1] agent_chat returned: 以上就是 `README.md` 的完整内容。核心信息概括：
**C Agent** 是一个纯 C 实现的轻量级终端编程智能体，上海交通大学 CS2313 操作系统课程设计项目。它基于 LLM 工具调用，具有：
```

Figure 6: SubAgent spawn/status/wait workflow

8 上下文 Offload 与 Summary

当上下文预算过大时，C-Agent 首先把大工具输出 offload 到 `.agent/offload/`。上下文中的 tool message 会被替换为短 preview 和恢复指令；如果对话仍然过长，summary policy 会把旧消息压缩为结构化 handoff。为了能 agent 未来能使用上 offload，还新增 manifest 给 agent 来知道什么被卸载了。

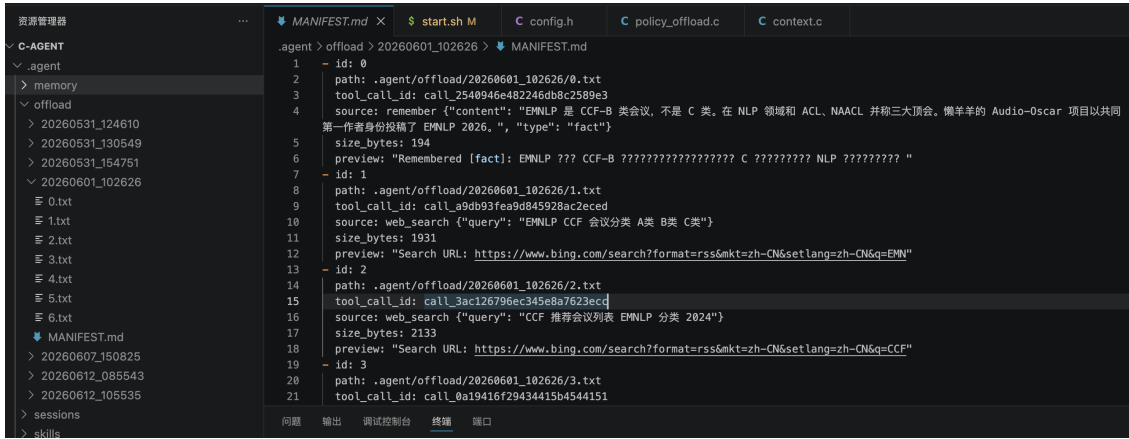


Figure 7: Offload manifest 与恢复指针

9 运行时安全与鲁棒性

C-Agent 允许 LLM 请求真实文件和真实命令，因此运行时必须拥有最终裁决权。我的实现不是简单相信模型输出，而是在工具边界处增加了多层防护：文件路径必须通过 workspace sandbox，危险 shell 命令会被拦截，联网工具禁止访问 localhost/private network，LLM 请求前还会清理非法 UTF-8，避免二进制输出破坏 JSON 协议。

机制	作用
Workspace sandbox	<code>resolve_workspace_path</code> 使用 <code>realpath</code> 解析真实路径，防止 <code>../</code> 、绝对路径和 <code>symlink</code> 逃逸。
危险命令过滤	<code>bash</code> 工具在 <code>fork</code> 前检查 <code>denylist</code> ，例如 <code>rm -rf /</code> 、 <code>fork bomb</code> 、 <code>raw disk write</code> 、 <code>curl sh</code> 等模式。
Private network blocking	<code>web_fetch</code> 只抓取外部 <code>http/https</code> URL，拒绝 <code>localhost</code> 和内网地址，避免 agent 被诱导访问本地服务。
UTF-8 清理	<code>llm_client.c</code> 在构造 JSON 请求前清理非法 UTF-8，防止工具输出中的二进制数据损坏 <code>wire protocol</code> 。
网络超时与重试	LLM 客户端设置超时和有限重试，避免临时网络错误直接破坏整轮对话。

10 Executor 调度与 UI 同步

Executor 的核心目标是同时满足性能和协议正确性：只读工具可以并发执行，但返回给 LLM 的 `tool messages` 必须严格保持 `request order`。如果 `batch` 中出现未知工具或写工具，调度器就回退到串行执行。这种策略保守、可审计，也避免了复杂依赖分析带来的不确定性。

Listing 2: Executor scheduling rule

```

if count >= 2 and every tool is registered and read_only:
    run all tasks concurrently
else:
    run tasks serially in request order

always:
    out_msgs[i] corresponds to tool_calls[i]
```

UI 侧也有一个类似 OS 同步问题：主线程和渲染线程都可能写终端。实现中让 `render thread` 拥有动态区域（spinner、工具状态），主线程只发送 UI event；在打印最终回复前，`ui_idle()` 作为 barrier，等待渲染线程释放终端输出权。这样避免 spinner 和最终回答互相打断。

```
zsh c-agent main i44 i1 73 ~1 0ms base 3.13.13 46% 12,10:54
./start.sh
260- char user_dir[PATH_MAX];
261- char project_dir[PATH_MAX];
262- user_skills_dir(user_dir, sizeof(user_dir));
... (3 more lines)
✓ read_file {"path":"memory.c","offset":698,"limit":50} (0.0s)
#include "memory.h"

#include "agent/llm_client.h"
#include "cJSON.h"
#include "config.h"
#include "message.h"
#include "util.h"

... (41 more lines)
✓ read_file {"path":"skills.c","offset":255,"limit":50} (0.0s)
#include "skills.h"

#include "config.h"
#include "util.h"

#include <ctype.h>
#include <dirent.h>
#include <errno.h>
... (41 more lines)
✓ read_file {"path":"tools/sandbox.c"} (0.0s)
/*
 * Workspace containment - see sandbox.h for the threat model.
 *
 * Why we use realpath(3) and not a string-level "does it contain ..":
 */
```

Figure 8: 只读工具并发执行，同时保持 request-order invariant

11 Hooks 与交互控制面

Hooks 系统把工具执行过程中的横切逻辑从 executor 中解耦出来。Executor 只在工具运行前后 emit 事件；memory observation、未来的 logging、metrics、audit 都可以作为 subscriber 挂上去，而不用修改调度核心。这一点和 OS 中常见的事件通知/tracepoint 机制很像。

Listing 3: Tool lifecycle hooks

```
HOOK_BEFORE_TOOL    // before execution, result is NULL
HOOK_AFTER_TOOL     // after execution, includes args and result
executor_run_tools(...)
-> hooks_emit(HOOK_BEFORE_TOOL, ...)
-> run tool
-> hooks_emit(HOOK_AFTER_TOOL, ...)
memory_tool_hook(...)
-> memory_observe(tool_name, args, output)
```

除了 LLM 自主行动外，C-Agent 还保留了用户可直接操作的 slash-command 控制面：/session 管理历史会话，/memory 整理工具观察，/remember 让主 Agent 主动保存对话知识，/model 查询/切换模型，/help 展示命令。这让系统不是一个完全不可控的黑盒，而是一个可以被用户运营和维护的 agent runtime。

```
static HookFn g_hooks[HOOK_EVENT_COUNT][MAX_HOOKS_PER_EVENT];
static int g_hook_counts[HOOK_EVENT_COUNT];
static pthread_mutex_t g_hooks_lock = PTHREAD_MUTEX_INITIALIZER;

int hooks_register(HookEvent event, HookFn fn) {
    if (event < 0 || event >= HOOK_EVENT_COUNT || !fn)
        return -1;
    if (g_hook_counts[event] >= MAX_HOOKS_PER_EVENT)
        return -1;
    g_hooks[event][g_hook_counts[event]++] = fn;
    return 0;
}

void hooks_emit(HookEvent event, const void *data) {
    if (event < 0 || event >= HOOK_EVENT_COUNT)
        return;
    pthread_mutex_lock(&g_hooks_lock);
    for (int i = 0; i < g_hook_counts[event]; i++)
        g_hooks[event][i](event, data);
    pthread_mutex_unlock(&g_hooks_lock);
}

static void memory_tool_hook(HookEvent event, const void *data) {
    if (event != HOOK_AFTER_TOOL || !data)
        return;
    const HookToolEvent *ev = data;
```

Figure 9: Hooks 将 memory observation 从 executor 中解耦出来


```

> /help
Commands:
/model          Interactive model picker
/session        Browse and restore saved sessions
/session new    Create a new session
/session name X Rename current session
/session delete X Delete a session
/session restore X Restore a session by id
/memory        Consolidate observed facts into project memory
/remember      Ask the main agent to save conversation memory
/help          Show this help
exit/quit/q    Exit
> /model
deepseek-chat
deepseek-v3.2
deepseek-reasoner
minimax
minimax-m2.7
glm
glm-5.1
qwen
qwen3.5-27b
qwen3coder
↑ move Enter select Esc cancel

```

Figure 10: Slash commands 构成用户可控的操作界面

12 Prompt 构造与注入

C-Agent 并不依赖一条写死的 instruction，而是由多个模块共同拼装 prompt。每个模块负责一个较窄的控制面：基础 Agent 行为、持久化 memory、skill discovery、上下文恢复，以及特殊用途的辅助 LLM 调用。本节直接复制实现中的关键 prompt 片段，并说明它们分别如何进入模型上下文。

Listing 4: system prompt 拼装路径

```

agent_create()
-> agent_build_system_prompt()
    -> base system template           // agent/prompt.c
    -> Current Time                   // agent/prompt.c
    -> memory_build_prompt()          // memory.c
    -> skills_build_prompt()          // skills.c
llm_chat()
-> prepend {"role":"system","content":system_prompt}
-> append conversation history
-> advertise registered ToolDef schemas

```

12.1 基础 Agent Prompt: agent/prompt.c

这是主 system prompt。运行时 %s 会被替换为当前 workspace 路径，后续动态 section 会继续追加到这个模板之后。

Listing 5: Core system prompt template used by C-Agent

```

You are a coding agent running in the CLI at %s.
Use the provided tools when you need to run shell commands.
Return a short, final text reply when the task is done.

Memory:
- Use the recall tool at the start of a task to check what you already know about
  this project.
- Use the remember tool proactively. Do not wait for the user to explicitly say '
  remember this' when information is clearly durable.
- Remember stable user facts: identity, role/year, school/lab/advisor, research
  direction, long-term goals, and explicit corrections.
- Remember user preferences: language, tone, answer length, workflow habits, naming/
  role conventions, and recurring requests.
- Remember project facts and workflows: build/test commands, environment
  requirements, tool setup, architecture decisions, and repeated bugs or fixes.
- Do not remember one-off command output, temporary errors, unverified search
  results, transient plans, or jokes unless the user confirms they matter.

```

- Good memory entries are specific and concise. Example: 'User is a SJTU sophomore in X-LANCE Lab working on speech/NLP.'
- Memory persists across sessions. What you remember now will help you in future conversations.

SubAgent:

- Use `subagent_spawn` to start child agents in the background for independent subtasks.
- Use `subagent_status` to check what subagents are running.
- Use `subagent_wait` to get a subagent's result when it's done.
- The child has its own context and runs autonomously - intermediate output stays out of your context.
- Good use cases: exploring a codebase, running multiple checks, searching for patterns.
- You can spawn multiple subagents for parallel work, then collect results.

Context management:

- Prior history may contain '[CONTEXT SUMMARY - prior conversation compressed]'. Treat it as the authoritative handoff for older turns.
- Tool outputs may contain '[OFFLOADED_TOOL_OUTPUT]' with a path under `.agent/offload/`. If your answer or next action depends on omitted content, call `read_file` on that path first.
- Do not guess from an offload preview when the full content is needed.

Web access:

- Use `web_search` when the user asks to search the web or find current information without giving a URL. Use `web_fetch` for explicit URLs or to inspect search results. If `web_search` returns no parseable results, do not repeatedly fetch Bing result pages or issue near-duplicate searches; try one clearly different query or say the result is uncertain. Do not fetch localhost or private-network URLs.
- Use `install_skill` only when the user asks to install a skill package. It installs `SKILL.md` packages without executing scripts.

Git commits:

- When creating git commits, include this trailer exactly: Co-authored-by: lazyegg-c-agent <lazyegg-c-agent@users.noreply.github.com>

12.2 动态追加：时间、Memory 与 Skills

在基础模板之后，`agent_build_system_prompt` 会追加三个运行时相关的 section。第一个是当前时间：

Listing 6: Current time section appended by `agent/prompt.c`

```
## Current Time
Current system time: <YYYY-MM-DD HH:MM:SS ZONE>
```

第二个来自 `memory_build_prompt`。它刻意只注入 `preference` 和 `fact` 两类 memory；完整 memory store 则通过 `recall` 工具按需查询。

Listing 7: Memory snippet shape produced by `memory.c`

```
## Project Memory
## Preferences
- <selected durable user preference>

## Facts
- <selected durable project/user fact>
```

第三个是 `skill manifest`。启动时只注入短 metadata；完整 `SKILL.md` 按需加载。

Listing 8: Skill manifest prompt from `skills.c`

```
## Available Skills
Load a skill with load_skill when it is relevant. Startup only includes this
manifest; load_skill returns the full SKILL.md.
Skill scopes: project skills override user skills with the same name.
- <skill-name> [project|user]: <one-line description> (allowed-tools: ...)
```

12.3 工具 Schema Prompt: `llm_client.c` 与 `tools/*.c`

工具定义本身也是一种 prompt channel。每个注册的 `ToolDef` 都会向 LLM 请求贡献工具名、描述和 JSON schema。也就是说，工具 schema 不只是机器接口，也是在告诉模型“有哪些动作可以做、每个动作该怎样传参”。

Listing 9: Tool schema shape advertised to the LLM

```
{
  "type": "function",
  "function": {
    "name": "<tool name>",
    "description": "<tool description>",
    "parameters": <JSON schema from ToolDef.param_schema>
  }
}
```

12.4 Summary Handoff Prompt: `context/policy_summary.c`

当旧历史需要压缩时，summary policy 会用一条独立的受限 system prompt 调用 LLM。这样 summary 生成就不会混同于普通 assistant 回复，而是专门生成结构化 handoff。

Listing 10: Summary handoff system prompt

```
You are a conversation handoff engine for a coding agent.
Compress the prior conversation into a structured handoff that the next assistant
can rely on.
Do not include generic commentary. Preserve concrete state only.
If an offloaded tool output marker appears, preserve its id/path and explain when it
must be recovered.
Use exactly these markdown sections when applicable:
## Goal
## Completed
## Current State
## Key Files
## Decisions
## Offloaded References
## Open Questions
## Next Steps
```

12.5 Offload Recovery Prompt: `context/policy_offload.c`

offload policy 会把大 tool message 改写成短 preview 加机器可读的恢复提示。这不是通过 system message 注入，而是通过历史消息本身注入恢复语义。

Listing 11: Offloaded tool-output placeholder

```
<short preview>
[OFFLOADED_TOOL_OUTPUT]
id: <id>
path: .agent/offload/<session_id>/<id>.txt
size_bytes: <original size>
```

```
manifest: .agent/offload/<session_id>/MANIFEST.md
recovery: call read_file with {"path": ".agent/offload/<session_id>/<id>.txt"} before
relying on omitted content.
```

12.6 Memory Consolidator Prompt: memory.c

/memory 命令使用一次独立的 helper LLM 调用。它的 system prompt 明确禁止调用工具，并要求只返回严格 JSON，比让主 Agent 自由改写 memory 文件更可控。

Listing 12: Restricted memory consolidator system prompt

```
You are a restricted memory consolidator for a coding agent.
You do not call tools. You only return strict JSON.
Be conservative: fewer high-value memories are better than noisy logs.
```

Listing 13: Memory consolidation user prompt shape

```
Observed tool activity from this session:
<observed tool activity>
Existing project memory:
<existing memory>

Decide which observations are durable, useful project memories.
Save only stable facts, user preferences, architecture decisions, build/test
workflows, or real bugs/fixes.
Do not save routine read_file/bash logs, transient command output, temporary
failures, or vague facts.
Return strict JSON only, with at most 5 items, in this exact shape:
{"memories":[{"type":"workflow|architecture|pattern|preference|bug|fact","content":
  "concise memory"}]}
If nothing is worth saving, return {"memories":[]}
```

12.7 Conversation Memory Prompt: cmd.c

/remember 命令把一条 user-level instruction 注入主对话。和 /memory 不同，它让普通 Agent 自己判断是否需要调用 remember 工具。

Listing 14: /remember prompt

```
Review our conversation so far. Use the remember tool to save any important durable
information about this project, the user's preferences, architecture decisions,
build/test workflows, key files, repeated bugs, or explicit corrections. Be
specific and concise. Call remember once for each distinct high-value memory. Do
not save routine command output, temporary plans, jokes, or anything the user
may not want stored. If nothing is worth saving, say so briefly.
```

13 当前限制

- 当前 token 估算仍是粗略 heuristic，不是真实 tokenizer。
- system prompt 在 Agent 创建时构建；会话中新增的 memory 不会自动重新注入当前 system prompt，但仍可通过 recall 查询，并会在后续 Agent 实例中生效。
- public baseline 的 tools_advertised 测试假设只有四个工具，而扩展版有意暴露了十六个工具，因此会出现接口预期不一致。